Pràctiques amb el TurtleBot

Teniu varies maneres de treballar amb el Robot. Una és fent els nostres programes en Python i executant-los des del nostre ordinador fent servir un "simulador del robot", o fent servir el robot físic del laboratori.

Veiem primer la versió simulada, que es la que ens interessa a la primera sessió de laboratori amb el Robot:

Simulació TurtleBot

Aquesta comanda prepara l'ordinador i el ROS2 per treballar amb la informació del robot

```
ros2@ros2-humble:~$ source turtlebot3_ws/install/setup.bash
```

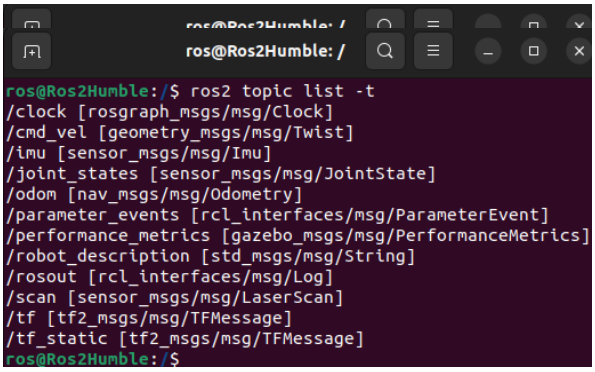
Arranqueu el Turtlebot simulat per que executi les comandes que li enviarem des del nostre programa:

```
ros2@ros2-humble: ~$ export TURTLEBOT3_MODEL=burger
ros2@ros2-humble: ~$ ros2 launch turtlebot3_gazebo turtlebot3_laberint.launch.py
```

Amb aquesta comanda s'arrenca el procés del simulador del Turtlebot, així com el visualitzador Gazebo. Així podrem visualitzar el nostre robot disposat al laberint.

Al igual que en el cas de treballar amb el robot físic, es poden visualitzar els tòpics que s'han arrencat amb la simulació del nostre robot:

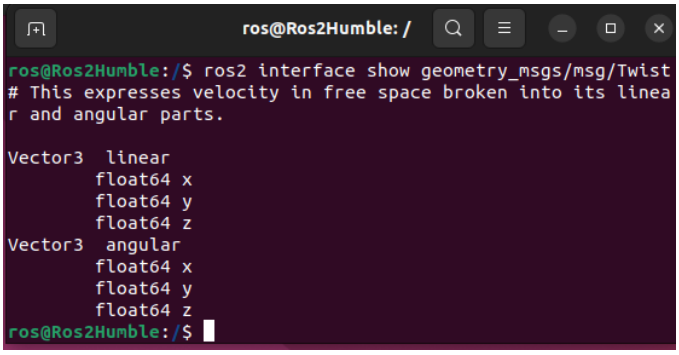
```
ros2@ros2-humble: ~$ ros2 topic list -t
```



Amb aquesta instrucció, veiem els "tòpics" que tenim disponibles del robot, i els tipus de missatge que es fan servir en cada tòpic. Per exemple, el tòpic *cmd_vel* fa servir un missatge que es diu *Twist*.

Per veure quin format de dades espera el missatge, podem fer servir la següent comanda:

```
ros2@ros2-humble: ~$ ros2 interface show geometry_msgs/msg/Twist
   /*Ens retorna el format/format de dades del missatge*/
```



Com podeu veure la informació del missatge Twist està format per 6 valors reals, que defineixen la velocitat lineal i angular del robot.

A partir d'ara ja podeu treballar al terminal de la vostra màquina amb les comandes de ROS2, o amb els vostres programes en Python.

Un primer programa senzill seria moure el nostre robot pel laberint de la nostre simulació.

Aquest programa treballarà amb el tòpic "**cmd_vel**", amb el que enviarem les comandes de velocitat (lineal i angular) al robot.

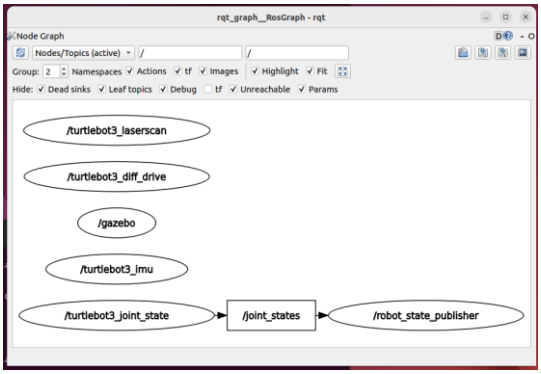
Comandes per Terminal:

Podem enviar el missatge des del terminal, per publicar el tòpic i moure el nostre robot. En aquest cas, li enviem/publiquem les velocitats amb totes les components a 0.0 menys la rotació respecte a l'eix Z:

```
ros2@ros2-humble: ~$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.04}}" &
                    /*enviem moviment angular a l'eix Z */
ros2@ros2-humble: ~$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" &
                    /*parem el moviment angular a l'eix Z */
```

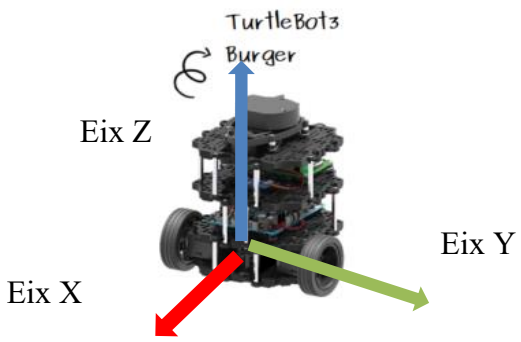
Podem visualitzar el graf de missatges que s'estan enviant els diferents nodes. Obrim un altre terminal i cridem a la comanda:

```
ros2@ros2-humble: ~$ ros2 run rqt_graph rqt_graph
```



Comandes per Programa:

Com hem comentat abans, podem treballar amb el robot fent servir els nostres programes en Python per connectar-nos i controlar el robot. Per això arrencarem un programa d'exemple que també treballarà amb el tòpic "cmd_vel". Es per això que el nostre programa té dos paràmetres d'entrada: **velocitat lineal** i **velocitat angular**. Tots dos paràmetres son de tipus real.



La velocitat lineal, és la corresponent a un desplaçament en l'eix X del robot (perpendicular a l'eix Y de les rodes), i la velocitat angular es correspon a una velocitat de rotació al voltant de l'eix Z del robot (perpendicular als eixos X i Y).

Exemple:

Rotació "pura" cap a l'esquerra: `./corre.py 0.0 0.1`
Rotació "pura" cap a la dreta: `./corre.py 0.0 -0.1`
Translació cap endavant: `./corre.py 0.05 0.0`
Translació cap enrere: `./corre.py -0.05 0.0`

Per parar el robot: Fer Ctrl-C dos cops per parar el programa.

Robot físic TurtleBot

Per provar la versió amb el robot físic, haurem d'arrencar el nostre robot i connectar-nos per ssh des de el nostre ordinador:

ros2@ros2-humble:~$ **ssh turtlebot@turtlebot1** **(Pot ser @turtlebot2)**
ros2@ros2-humble:~$ **Password: ros**

ros2@ros2-humble:~$ **export ROS_DOMAIN_ID = 31** **(o 30)**

I arranqueu el Programari que controla al Turtlebot del robot físic, **en el terminal corresponent al robot**, per que executi les comandes que li enviem des del nostre programa.

turtlebot@turtlebot1: ~$ **ros2 launch turtlebot3_bringup robot.launch.py**

(Proveu **~$ ros2 topic list -t** en qualsevol dels terminals, per comprovar que el robot està en marxa. Hauria de sortir la mateixa informació en els dos terminals.)

A partir d'ara, ja podem cridar els programes en Python a la vostra màquina, i es comunicaran amb el "Turtlebot físic" per tal d'executar les diferents comandes.

Com exemple, anem a moure el nostre robot al escenari amb els mateixos programes que hem fet en la versió simulada:

Enviant comandes en ros2:

ros2@ros2-humble:~$ **ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.04}}" &**
/*enviem moviment angular a l'eix Z */

O treballant en python:

Rotació "pura" cap a l'esquerra: **ros2@ros2-humble:**~$ **./corre.py 0.0 0.1**
Rotació "pura" cap a la dreta: **ros2@ros2-humble:**~$ **./corre.py 0.0 -0.1**
Translació cap endavant: **ros2@ros2-humble:**~$ **./corre.py 0.01 0.0**
Translació cap enrere: **ros2@ros2-humble:**~$ **./corre.py -0.01 0.0**

Introducció a la pràctica 2:

En aquesta pràctica treballarem sobretot amb la informació que ens aporta el sensor LIDAR. Aquest sensor ens proporciona distancies a objectes que estiguin als voltants del nostre robot.

Per treballar amb aquest sensor, haurem de connectar-nos als missatges que publica i fer servir la informació que proporciona.

Si veiem la llista de tòpics publicats a la nostre màquina:

ros2@ros2-humble: ~$ **ros2 topic list - t**

```
ros@Ros2Humble: ~
ros@Ros2Humble:~$ ros2 topic list -t
/clock [rosgraph_msgs/msg/Clock]
/cmd_vel [geometry_msgs/msg/Twist]
/imu [sensor_msgs/msg/Imu]
/joint_states [sensor_msgs/msg/JointState]
/odom [nav_msgs/msg/Odometry]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/performance_metrics [gazebo_msgs/msg/PerformanceMetrics]
/robot_description [std_msgs/msg/String]
/rosout [rcl_interfaces/msg/Log]
/scan [sensor_msgs/msg/LaserScan]
/tf [tf2_msgs/msg/TFMessage]
/tf_static [tf2_msgs/msg/TFMessage]
```

Podem veure el tòpic /scan que correspon al sensor lidar. Veiem el format de missatge que ens proporciona:

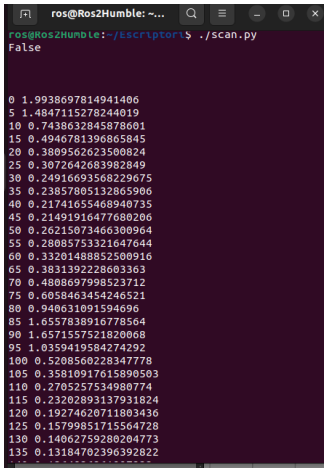
ros2@ros2-humble: ~$ **ros2 interface show show sensor_msgs/msg/LaserScan**

```
ros@Ros2Humble: ~
un', 'security', 'service', 'topic', 'wtf')
ros@Ros2Humble:~$ ros2 interface show sensor_msgs/msg/LaserScan
# Single scan from a planar laser range-finder
#
# If you have another ranging device with different behavior (e.g. a sonar
# array), please find or create a different message, since applications
# will make fairly laser-specific assumptions about this data

std_msgs/Header header # timestamp in the header is the acquisition time of
        builtin_interfaces/Time stamp
                int32 sec
                uint32 nanosec
        string frame_id
                                # the first ray in the scan.
                                #
                                # in frame frame_id, angles are measured around
                                # the positive Z axis (counterclockwise, if Z is up)
                                # with zero angle being forward along the x axis

float32 angle_min               # start angle of the scan [rad]
float32 angle_max               # end angle of the scan [rad]
float32 angle_increment         # angular distance between measurements [rad]

float32 time_increment          # time between measurements [seconds] - if your scanner
                                # is moving, this will be used in interpolating position
                                # of 3d points
float32 scan_time               # time between scans [seconds]

float32 range_min               # minimum range value [m]
float32 range_max               # maximum range value [m]

float32[] ranges                # range data [m]
                                # (Note: values < range_min or > range_max should be discarded)
float32[] intensities           # intensity data [device-specific units].  If your
                                # device does not provide intensities, please leave
                                # the array empty.
```

De tota la informació que publica el Lidar, nosaltres farem servir la taula **ranges**, que ens proporciona les distàncies als objectes al voltant del robot en metres.

A l'escriptori tenim un programa que ens permet imprimir per pantalla els valor d'aquesta taula en increments de 5 graus:

ros2@ros2-humble: ~Escriptori$ ./scan.py



Podem veure els valors que es van generant contínuament del LIDAR amb els rebots dels objectes del laberint (parets).

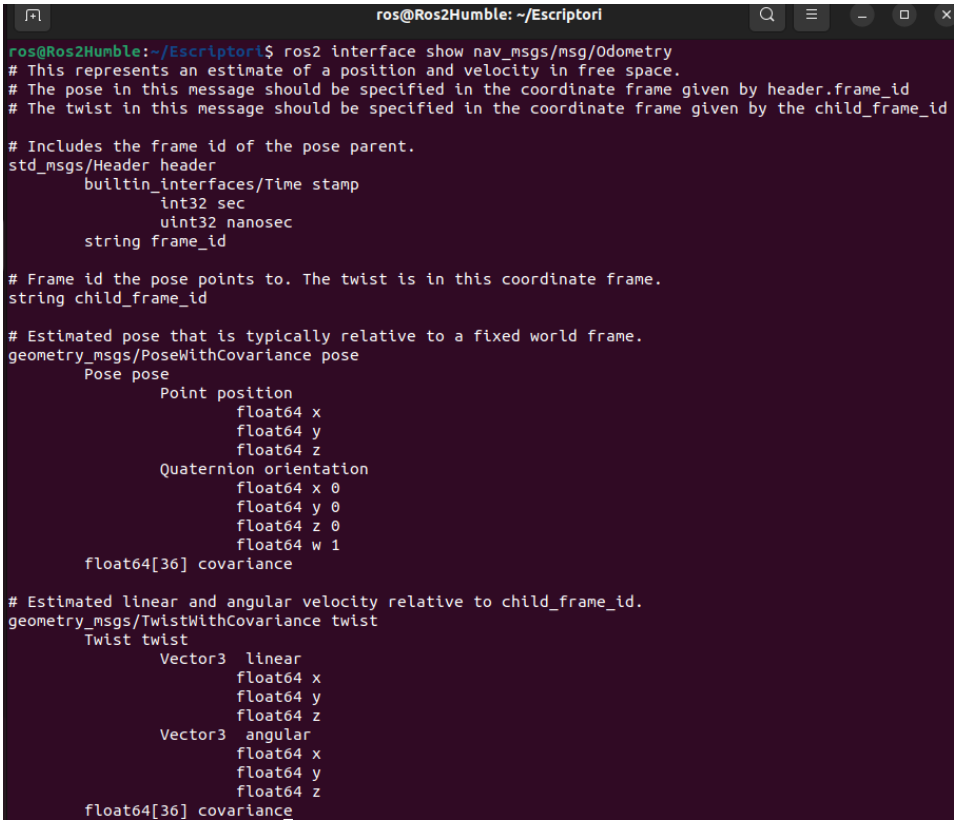
El valor de la primera columna es correspon a l'angle de visualització (0 davant, 90 esquerra, 180 darrera i 270 dreta).

I el de la segona columna és la distancia del sensor a l'objecte, que és un valor real i el tenim expressat en metres.

També podem veure veure el tòpic **/odom** que correspon a la informació de posició i velocitat del nostre robot, tenint en compte únicament els motors de les rodes i el seu desplaçament.

Veiem quin format de missatge tenim:

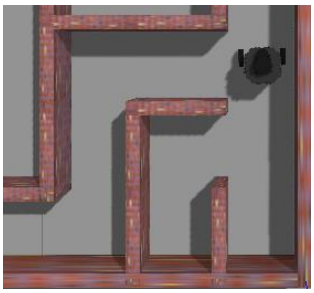
ros2@ros2-humble: ~$ ros2 interface show show nav_msgs/Odometry



i si volem veure el que està publicant instantàniament el nostre tòpic, podem cridar a la comanda:

```
ros2@ros2-humble: ~$ ros2 topic echo /odom
```

Ara, que ja coneixem els dos tòpics nous que treballarem a la pràctica 2, anem a situar el nostre robot la posició de la imatge (aprox) i veiem la informació que ens proporcionen:



Comprovem:

1.- la posició del robot que ens aporta la odometria (x i y).

```
ros2@ros2-humble: ~$ ros2 topic echo /odom
```

2.- la informació que ens aporta el LIDAR.

```
ros2@ros2-humble: ~$ ./scan.py
```

Ara ja podem passar a veure/treballar amb la pràctica 2.

Pràctica 2:

En aquesta pràctica el que pretenem es treure el nostre robot del Laberint. Per aconseguir realitzar la tasca farem servir la informació que ens aporta el sensor LIDAR que ens proporciona distancies a objectes que estiguin als voltants del nostre robot.

El LIDAR és un sensor circular molt interessant, perquè aporta informació en 360 graus del robot. Per això haurem de entendre que el sensor només ens dona una informació de distancia als objectes, però no identifica res més. No podem saber quin tipus d'objecte és. No és un sensor "visual" del que podríem extreure característiques per discriminar els diferents objecte del mon.

Amb al LIDAR a única informació que tenim és una taula de valors reals que ens indican la distancia en metres dels "ecos" dels objectes. Aquesta taula té 360 posicions, o sigui que la resolució espacial del sensor és de 1 grau.

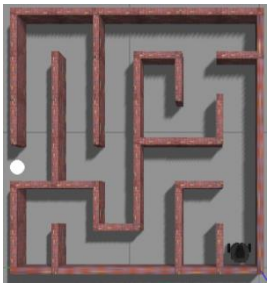
Com que tenim el robot ubicat en un laberint, els únics objectes que ens donen ecos son les parets del laberint. Però tenim la taula de ecos completament plena, perquè el robot està pràcticament rodejat de parets en tot el seu recorregut.

Tasca a realitzar:

Desenvolupar un programa en python, que fent servir la informació que ens proporciona el LIDAR, vagi enviant comandes de moviment al robot **de forma automàtica** per sortir del laberint.

Com que no tenim mes informació del laberint i no sempre s'encerta quan agafem un camí, haureu de tenir en compte com guardar un "mapa del camí" recorregut per si heu de fer marxa enrere i agafar alguna altre camí alternatiu al llarg del recorregut.

El laberint "real" que tenim definit és el que podeu veure a la imatge. El Robot està situat a la posició inicial del laberint i la sortida del laberint la marquem amb una taca blanca al terra.



Aquesta marca serà detectada pel robot físic amb un sensor LED, situat sota del robot, que ens proporciona informació booleana. El sensor es posarà a *True* quan el robot estigui situat a sobre de la taca i la detecti. Mentre que al llarg del camí, aquest sensor estarà desactivat (*False*).

En el cas de fer servir la simulació del robot amb el Gazebo, per fer la simulació del sensor LED farem servir l'odometria del robot simulant el final del laberint amb el càlcul d'una distancia a un punt determinat de l'escena. Es a dir, s'activarà el "sensor de final" quan el robot estigui a una distancia més petita que un llindar determinat. I ens donarà *False* mentre el robot estigui a una distancia més gran.

Podem veure el funcionament del sensor LED real (i el simulat) fent una ullada als programes: **meta_real.py** i **meta_sim.py.**

meta_real.py :

En aquest llegim la informació que ens proporciona una de les entrades digitals que tenim en el nostre robot, i que la publica el sensor LED. Quan arribem a la marca de final_sortida del laberint, el sensor LED publicarà un *True* al nostre programa.

meta_sim.py :

En aquest fem servir la odometria del robot per calcular la posició instantània del robot al llarg del recorregut. Amb aquesta informació podem calcular la distancia del robot a la posició de sortida del laberint.

Aquesta posició, la hem definit nosaltres. Però la podeu comprovar, portant el robot manualment (amb el programa que vàreu fer a la primera pràctica per controlar el robot amb el teclat) fins a la sortida i veient la publicació del tòpic **/Odometry** en el moment que esteu a la sortida del laberint.